

Success Library Artifacts Guide

This is your reference sheet. Use it to answer questions about what we've built, why each piece exists, and how they connect. If someone asks "what is this file?" or "why do we need that?" - the answer is here.

The Big Picture

We are building a governed system to answer three questions for every marketing campaign:

1. What counts as success? (the definition)
2. How do we calculate it? (the code)
3. Where does the data live? (the source)

Today these answers are scattered across SAS scripts, tech specs, Confluence pages, and analyst notebooks. The Success Library centralizes them into a single framework that engineers can build a curated dataset from.

The Artifacts

1. Metric Code Registry

File: metadata/metric_code_registry.csv

What it is: The master manifest of every campaign and its success metric. One row per campaign-metric pair. This is the single source of truth for what exists, what's ready, and what still needs work.

Why it exists: Without this, there's no way to answer "how many campaigns are done?" or "what's left?" at a glance. It's the tracking sheet for the entire library build-out.

Key columns:

- metric_id - the governed ID that links everything together (e.g., VVD_ACQ_001)
- code_status - is the logic written? (active / stub / not_created)
- discovery_status - have we found the SAS code? (complete / pending_sas_search / missing_definition)
- code_ref - path to the code file in the repo

- next_action - what needs to happen next for this campaign

When someone asks: "How many campaigns are engineering-ready?" - filter code_status = active. Currently 6 VVD campaigns (10 rows counting secondary metrics). The rest are pending SAS extraction or missing definitions entirely.

2. Success Events Schema

File: metadata/vvd_success_events.csv

What it is: A sample of what the target success_events table looks like when populated with real VVD data. This is the schema the engineers will build - the output table that holds every success event across all campaigns.

Why it exists: The whiteboard defined the schema conceptually. This makes it concrete with actual column values, JSON structures, and event type codes. Engineers can look at this and know exactly what table to build.

Key columns (matching the whiteboard Table 1 green):

- data_asset - what type of event (card_acquisition, card_usage, etc.)
- clnt_no - client number
- lob, prod, sub_prod - the product taxonomy
- event_dt - when the success happened
- event_attributes - JSON column for flexible metadata (status codes, BINs, transaction types)
- event_type_cd - the numeric code linking to success_event_mapping
- mnemonic - which campaign this event belongs to
- success_level - 1 = primary, 2 = secondary

When someone asks: "What does the data actually look like?" - open this file. It has 24 sample rows across all 4 VVD event types and all 6 campaigns.

3. VVD Data Dictionary

File: docs/14_VVD_DATA_DICTIONARY.md

What it is: The complete technical reference for VVD. Every source table, every column, every filter, every business rule, every join condition. This is the deep-dive document.

Why it exists: The vintage engine has all this logic hardcoded in Python. This document extracts it into a format that doesn't require reading Python code to understand. It's the specification that an engineer or analyst can read to understand exactly how VVD success is detected.

What's in it:

- Architecture overview (how the 4 layers map to the vintage engine modules)
- Campaign-to-metric mapping with a reuse matrix
- Full column-level documentation for all 5 source tables (3 HIVE, 2 EDW)
- All 4 success metric definitions with filters and output contracts
- Experiment population schema (how clients are assigned to experiments)
- Output schemas (vintage curves, channel breakdown, client-level)
- Entity relationship diagram
- Business rules (success window, client extraction, engagement denominators)
- Gap analysis between the vintage engine and the target architecture

When someone asks: "How does wallet provisioning work?" or "What filters define card usage?" - this is where you point them. Section 4 has the answer for every metric.

4. VVD Success Events Schema Doc

File: docs/15_VVD_SUCCESS_EVENTS_SCHEMA.md

What it is: The schema documentation that explains the success_events table design - column definitions, event type codes, JSON structures, filter logic, and how it connects to campaign_mapping.

Why it exists: The CSV (artifact #2) shows the data. This doc explains the design decisions. Why is open_amt null for acquisitions? Why does wallet provisioning use AMT1 = 0? What goes in event_attributes? This document answers those questions.

When someone asks: "Why did you structure it this way?" - this doc has the rationale.

5. Code Files (Logic Repo)

Location: code/VVD/, code/CC/, code/MTG/

What they are: The actual SQL and PySpark code for each success metric. One file per metric. This is what the whiteboard calls "Layer 3 - GitHub Logic Repo."

Why they exist: This is the deliverable. When the curated dataset is built, these code files are what run inside the pipeline. Each file has:

- Metadata (product, pillar, campaigns using it, grain, source)
- Business definition (plain language)

- Filter logic table
- PySpark code (for HIVE)
- SQL code (for Data Warehouse)
- Integration notes (how it connects to the vintage engine)

Current status:

- code/VVD/ - 4 files, all active with production-validated code
- code/CC/ - 1 file, stub (template only, logic TBD)
- code/MTG/ - 1 file, stub (template only, logic TBD)

When someone asks: "Show me the code for card acquisition" - open code/VVD/VVD_ACQ_001.md.

6. SAS Search Pipeline

Location: sas_search/

What it is: The discovery tooling. PowerShell scripts that scan the network for SAS files containing campaign mnemonics, extract the code blocks around success logic, and feed them to Helios Assist for classification.

Why it exists: There are ~40 campaigns that have success logic buried somewhere in SAS scripts across the organization. We can't define the library metrics without first finding the existing logic. This pipeline automates that discovery.

Key files:

- keyword_mapping.csv - master list of all 46 campaigns with search keywords
- keyword_mapping_final.csv - campaigns grouped by product for batch processing
- scripts/ - the 7-step pipeline (scan -> tag -> extract -> split -> summarize)
- prompts/ - Helios Assist prompts for AI-assisted code extraction
- SUCCESS_LIBRARY_GUIDE.md - full user guide for running the pipeline

When someone asks: "How do we add a new campaign?" - add a row to keyword_mapping.csv and re-run the pipeline. The guide covers it.

7. Table Data Assets (Whiteboard Schemas)

File: docs/11_TABLE_DATA_ASSETS_MVD.md

What it is: The formalized version of the whiteboard session (2026-01-31). Documents all the target table schemas: experiment_mapping (A), experiment_population (B), campaign_mapping (C), success_event_mapping (1), success_events (1 green), tactic_history (2), cross_sell (5), Views 3 and 4, Channel Interactions, and Share of Wallet tables.

Why it exists: This is the architecture blueprint. Everything else we build must conform to these schemas. The engineers will use this to build the physical tables.

When someone asks: "What's the target data model?" - this is the answer.

8. Project Context

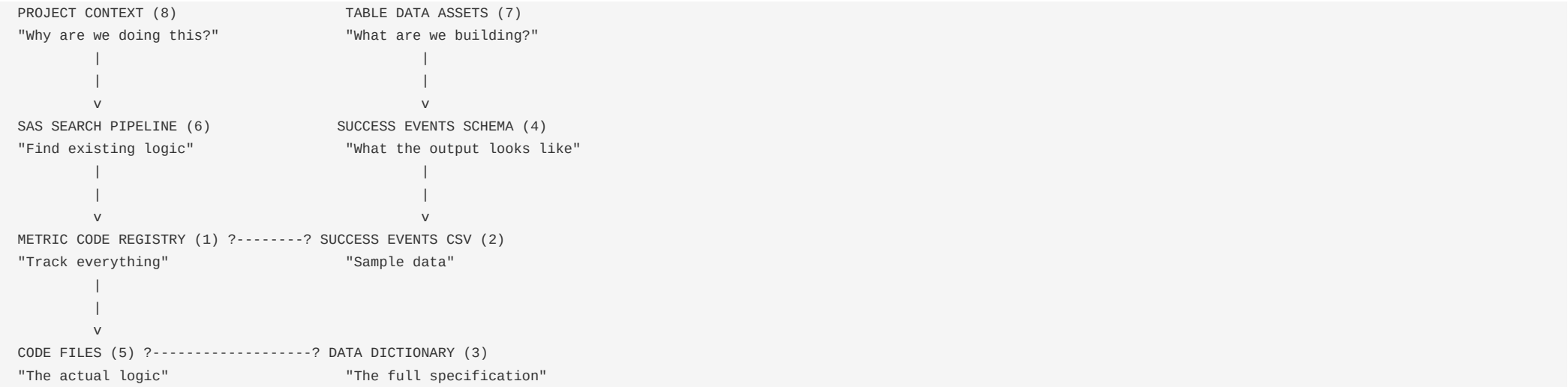
File: support/success_library_project_context.md

What it is: The full narrative - four-layer architecture, current state vs future state, tech roadmap, timeline, impact areas, code patterns, and open questions. This is the strategic document.

Why it exists: Anyone new to the project reads this first. It explains the problem, the solution, and the plan.

When someone asks: "What is the Success Library?" - start here.

How the Artifacts Connect



The flow:

- 1. Project Context explains the problem and architecture
- 2. Table Data Assets defines the target schemas
- 3. SAS Search Pipeline discovers existing success logic from SAS code
- 4. Metric Code Registry tracks what's been found and what's ready
- 5. Code Files contain the formalized, standardized logic per metric
- 6. Data Dictionary documents every detail of the code and source tables
- 7. Success Events Schema shows what the final data looks like
- 8. Engineers take (5), (6), and (7) to build the curated dataset

Status Summary

Category	Count	Notes
Total campaigns (MVP scope)	46	All in keyword_mapping.csv
Engineering-ready (code active)	6	VCN, VDA, VDT, VUI, VUT, VAW
Code stubs (template only)	3	PCQ/VBA/VBA_LTA (CC), FTH (MTG)
Pending SAS extraction	32	In pipeline, waiting for discovery
Missing success definition	5	MFY, MMT, RCR, COR, ZRR/VSX
Unique metric definitions needed	~25	Many campaigns share metrics

Common Questions and Where to Point People

Question	Artifact	Section
"What is the Success Library?"	Project Context (8)	Executive Summary
"What's the target data model?"	Table Data Assets (7)	Full document
"How many campaigns are done?"	Metric Code Registry (1)	Filter by code_status
"Show me the VVD success logic"	Code Files (5)	code/VVD/*.md
"What tables does VVD use?"	Data Dictionary (3)	Section 3: Source Data Assets
"What does the output data look like?"	Success Events CSV (2)	Open in Excel
"How do we add a new campaign?"	SAS Search Guide (6)	"Adding New Campaigns" section
"What are the business rules?"	Data Dictionary (3)	Section 8: Business Rules
"What's the gap between now and target?"	Data Dictionary (3)	Section 9: Open Questions
"What's the before/after impact?"	Project Context (8)	Current State vs Future State
"How does experiment tracking work?"	Table Data Assets (7)	Tables A and B
"What's the tech roadmap?"	Project Context (8)	Tech Stack Roadmap

What's Next

1. Continue SAS extraction - run the pipeline for the next product group (Credit Cards or Loans are the largest batches)
2. Formalize extracted logic - as Helios returns success code, create code files and update the registry
3. Resolve the 5 MISSING campaigns - get business owners to define success for MFY, MMT, RCR, COR, ZRR/VSX or mark them out of scope
4. Engineering handoff - package the registry + success_events schema + code files as the build spec for the curated dataset
5. Colibra integration - once the curated dataset is live and the library has 25+ active metrics, migrate the registry into Colibra for governed catalog management

This document is your cheat sheet. Keep it updated as artifacts evolve.